

BIO：同步阻塞，一个连接对应一个线程

- 客户端有连接请求时，服务器端就需要启动一个线程进行处理
- 如果这个连接不做任何事情，会造成不必要的线程开销，当然可以通过线程池机制改善
- 适用于连接数目比较小且固定的架构，对服务器资源要求比较高，并发局限于应用中

NIO：同步非阻塞，使用多路复用器，使单线程能够处理多个连接请求

- 客户端发送的连接请求，都会注册到多路复用器上，多路复用器轮询到连接有 I/O 请求时才启动一个线程进行处理
- 适用于连接数目多且连接比较短（轻操作）的架构，比如聊天服务器，并发局限于应用中，编程比较复杂
- 这个并不是真正意义上的异步框架，本质还是一个同步框架 + 多路复用，只是在进行读写操作的时候是非阻塞的

AIO：异步非阻塞

- 客户端的 I/O 请求都是由 OS 先完成了，再通知服务器应用去启动线程进行处理
- 适用于连接数目多且连接比较长（重操作）的架构，比如相册服务器，充分调用 OS 参与并发操作，编程比较复杂

	BIO	NIO	AIO
客户端	Socket	SocketChannel	AsynchronousSocketChannel
服务端	ServerSocket	ServerSocketChannel	AsynchronousServerSocketChannel
特点	1. 面向流 Stream，效率低	1. 面向缓冲区 Buffer（块），效率高	
	2. 需要拷贝到直接内存中，效率低	2. 直接操作直接内存	
	3. 阻塞	3. 非阻塞 Channel 可以实现异步操作	
	4. 多线程	4. 多路复用 Selector 可以实现多路复用	

概述

BIO

BIO：同步阻塞 I/O 模型，是传统的 I/O 模型

- 当执行 I/O 操作时，如果 I/O 操作未完成，线程会一直阻塞在那里，无法执行其他任务
- 在 BIO 中通常 "一个连接对应一个线程"
- BIO 的实现比较简单，java.io 包中的类：InputStream、OutputStream、ServerSocket、Socket 等，都属于 BIO

BIO 优点

- 实现简单，易于理解
- 适合 连接数少且稳定 的场景

BIO 缺点

- 并发处理能力弱，需要使用多线程/线程池进行改善
- 资源浪费严重

BIO 应用场景

- 文件读写
- 网络通讯
- 数据库操作

```
public class Server {  
  
    public static void main(String[] args) throws Exception {  
        // 创建一个ServerSocket, 并绑定到端口 8080  
        ServerSocket serverSocket = new ServerSocket(8080);  
  
        while (true) {  
            // 等待客户端连接  
            Socket socket = serverSocket.accept();  
  
            // 创建一个新的线程来处理客户端连接  
            new Thread(() → {                                // 一个  
连接对应一个线程  
                try {  
                    // 从客户端读取数据  
                    InputStream inputStream =  
socket.getInputStream();
```

```

        byte[] bytes = new byte[1024];
        int len = inputStream.read(bytes);

        // 将数据打印到控制台
        System.out.println(new String(bytes, 0,
len));

        // 向客户端发送数据
        OutputStream outputStream =
socket.getOutputStream();
        outputStream.write("Hello,
world!".getBytes());

        // 关闭连接
        socket.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}).start();
}
}
}

```

NIO

NIO：同步非阻塞 的 I/O 模型，是新型的 I/O 模型

- 当执行 I/O 操作时，如果 I/O 操作未完成，线程不会阻塞在那里，而是继续执行其他任务

- BIO 的实现比较复杂，java.nio 包中的类：Channel、Selector、Buffer 等

NIO 优点

- 并发处理能力强，浪费资源少
- 适合 连接数多且不稳定 的场景

NIO 缺点

- 实现复杂，不易理解

NIO 应用场景

- 即时通讯
- 网络聊天
- 游戏服务器

NIO 优化

- 多线程：使用多线程处理多个客户端请求
- 缓冲区：减少内存的分配与释放，提高 I/O 操作效率

NIO 允许开发者以更加高效的方式去处理 I/O 操作（非阻塞、多路复用）

```
public class Server {  
  
    public static void main(String[] args) throws Exception {  
        // 创建一个ServerSocketChannel, 并绑定到端口 8080
```

```
ServerSocketChannel serverSocketChannel =
ServerSocketChannel.open();
    serverSocketChannel.configureBlocking(false);
    serverSocketChannel.bind(new
InetSocketAddress(8080));

    // 创建一个Selector
    Selector selector = Selector.open();

    // 将ServerSocketChannel注册到Selector上, 并监听OP_ACCEPT事件
    serverSocketChannel.register(selector,
SelectionKey.OP_ACCEPT);

    while (true) {
        // 阻塞等待事件发生
        // 😊 Selector.select() 虽然是一个阻塞方法, 但这并不意味着 NIO
是一个阻塞式 I/O
        // 因为 Selector.select() 它是阻塞等待事件发生, 而不是阻塞等待
I/O 操作完成
        // 如果没有任何事件发生, 那么 Selector.select() 会阻塞线程, 直
到有事件到来
        // 但是如果一旦有事件发生, 那么 Selector.select() 会立刻返回
        selector.select();

        // 遍历所有已就绪的SelectionKey
        Set<SelectionKey> selectedKeys =
selector.selectedKeys();
        for (SelectionKey selectedKey : selectedKeys) {
            // 判断事件类型
            if (selectedKey.isAcceptable()) {
                // 客户端连接
```

```

        SocketChannel socketChannel =
serverSocketChannel.accept();
        socketChannel.configureBlocking(false);
        // 将SocketChannel注册到Selector上, 并监听
OP_READ事件

        socketChannel.register(selector,
SelectionKey.OP_READ);

    } else if (selectedKey.isReadable()) {
// 客户端发送数据

        SocketChannel socketChannel =
(SocketChannel) selectedKey.channel();
        // 读取数据
        ByteBuffer buffer =
ByteBuffer.allocate(1024);
        int len = socketChannel.read(buffer);

        System.out.println(new
String(buffer.array(), 0, len));

        // 将SocketChannel重新注册到Selector上, 并监听
OP_READ事件

        socketChannel.register(selector,
SelectionKey.OP_READ);
    }
}

// 清除已处理的SelectionKey
selectedKeys.clear();
}
}
}

```

AIO

AIO：异步的 I/O 模型，也被称为 NIO2.0，允许开发人员编写 "高并发的 I/O 应用"

- 在 AIO 中 Channel 接口变成了 **AsynchronousChannel**：
AsynchronousSocketChannel、**AsynchronousFileChannel** 都是它的子类
- 当 I/O 操作完成后，通过 **CompletionHandler** 回调函数或者 **Future** 来通知应用程序

AIO 适合处理那些 需要大量并发连接、I/O 操作相对耗时 的操作

- 相册服务器
- 数据库服务器

```
public class Server {  
  
    public static void main(String[] args) throws Exception {  
        // 创建一个AsynchronousSocketChannel, 并绑定到端口 8080  
        AsynchronousSocketChannel serverSocketChannel =  
AsynchronousSocketChannel.open();  
        serverSocketChannel.configureBlocking(false);  
        serverSocketChannel.bind(new  
InetSocketAddress(8080));  
  
        // 创建一个CompletionHandler
```



```
        CompletionHandler<Integer, Void> completionHandler =
new CompletionHandler<Integer, Void>() {
    @Override
    public void completed(Integer result, Void
attachment) {
        // 处理 I/O 操作完成后的结果
    }

    @Override
    public void failed(Throwable exc, Void
attachment) {
        // 处理 I/O 操作失败后的结果
    }
};

// 异步接受新的连接
serverSocketChannel.accept(null, completionHandler);

while (true) {
    // 等待 I/O 操作完成
    Thread.sleep(1000);
}
}
```

Buffer：直接内存（数据载体）

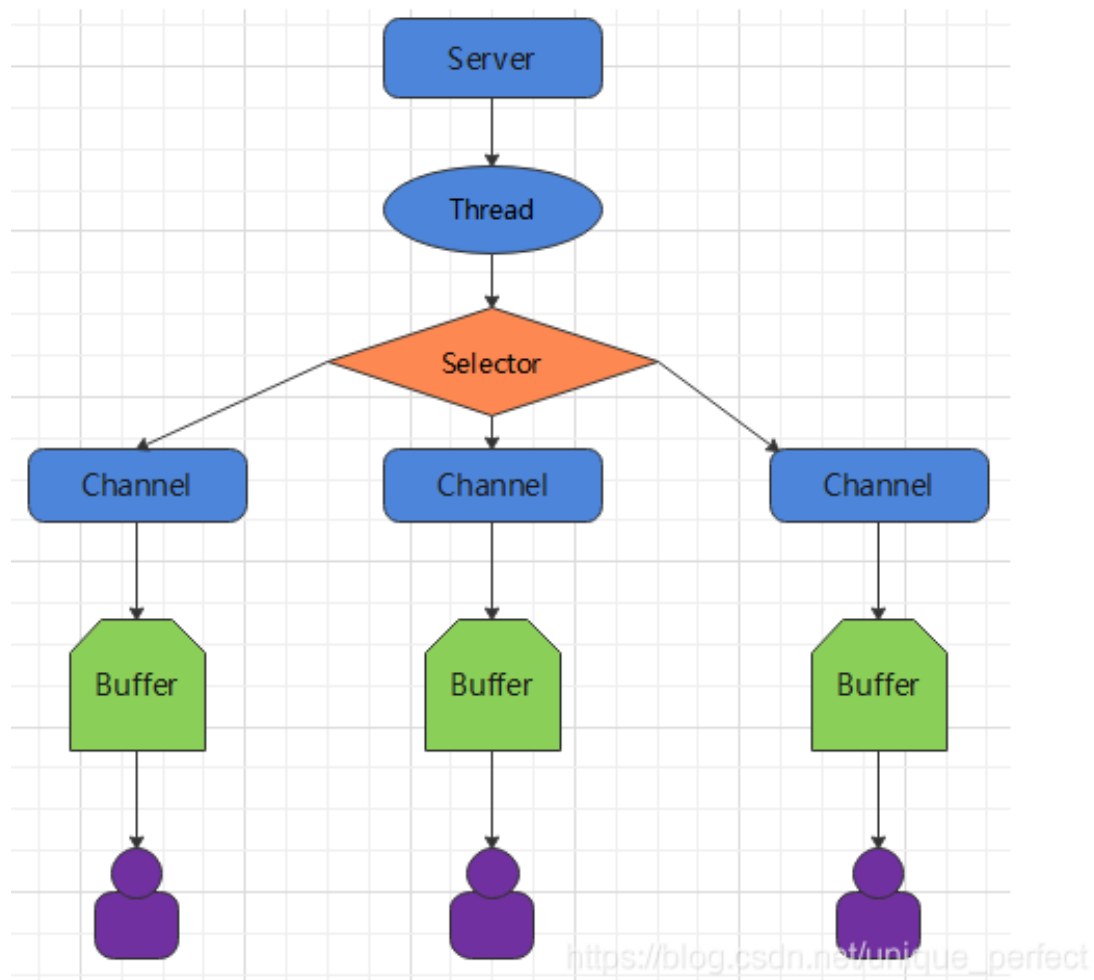
- flip()
- put()
- get()

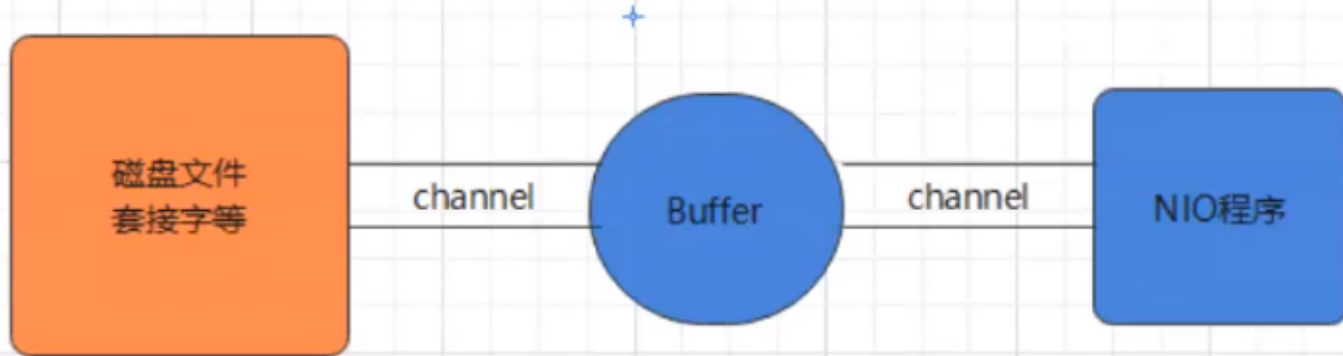
Channel：传输通道

- read()
- write()

Selector：多路复用

- select()
- selectKeys()





Buffer 内存

直接内存

Java Heap、Native Heap、Direct Buffer 三者的区别

1. Java 堆 (Java Heap) :

定义：JVM 虚拟机为 Java 代码分配的内存，用于存放 "**Java 对象实例**"

功能：Java Heap 由 **JVM 虚拟机** 进行分配与释放、由 **垃圾回收器** 进行管理

2. Native 堆 (Native Heap) :

定义：操作系统为原生代码（C/C++）分配的内存，用于存储**"原生数据"**

功能：我们可以直接通过 `malloc()/free()` 来进行分配与释放，不受 JVM/Java GC 管理

3. **直接内存 (Direct Buffer)**：

定义：由 **NIO** 库直接在操作系统层面上分配的 native 堆内存，用于**"高效的数据传输"**（避免 Java 堆和 Native 堆的频繁交互）

功能：由 **NIO** 库进行分配与释放，例如 `DirectByteBuffer/MappedByteBuffer`

直接内存的优缺点

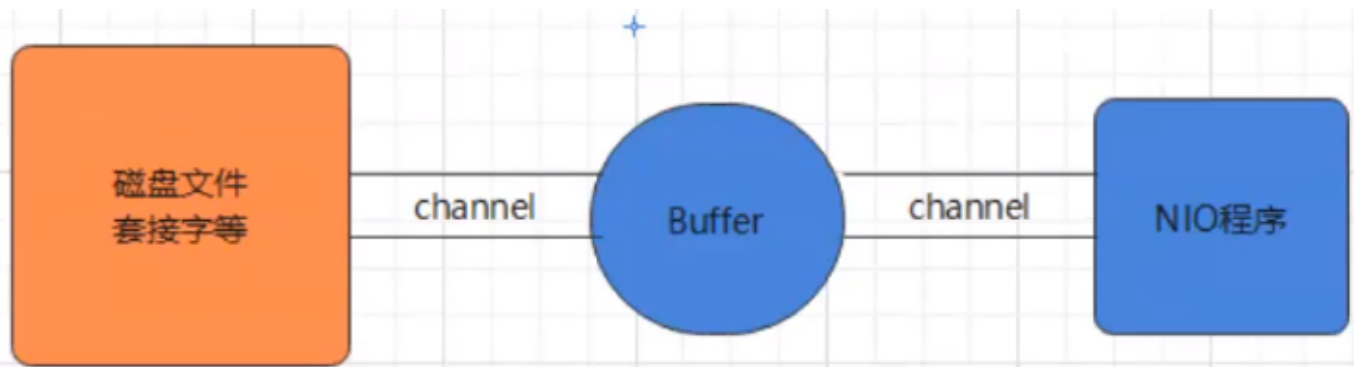
1. **效率高**：直接与 native 层通讯，因此在做 IO 处理时，比如网络发送大量数据时，直接内存会具有更高的效率
2. **不占有应用内存**：直接内存是在 JVM 之外申请的，因此它不会占用应用的内存
3. **创建比较消耗资源**：直接内存使用 `allocateDirect()` 创建，但是它比申请普通的堆内存需要耗费更高的性能

直接内存的使用场景

1. **大数据、高内存场景**：当你有很大的数据要缓存，并且它的生命周期又很长（需要更多的内存空间），那么就比较适合使用

直接内存

2. 频繁的 IO 操作：适合频繁的 IO 操作，比如网络并发场景（需要高效率），例如 OkHttp 就是使用的 NIO
3. 如果不是能带来很明显的性能提升，还是推荐直接使用堆内存（创建比较消耗资源）
4. 一般来说 NIO 中的 Channel 并不直接参与内存操作，而是通过 Buffer 直接内存间接操作



Buffer 缓冲区

Buffer 本地缓存数组

1. **position**：读写的当前位置
2. **capacity**：缓冲区的最大容量，一旦创建后不可更改
3. **limit**：剩余多少容量

写模式下，表示能往 Buffer 中写入多少数据，此时
 $\text{limit} = \text{capacity}$

读模式下，表示能从 Buffer 中读取多少数据，此时

$\text{limit}=\text{position}$

4. **mark**:

当调用 `reset()` 的时候，会将当前位置 $\text{position}=\text{mark}$ ，然后从 `mark` 处开始读写

常用方法

1. **Buffer clear()**: 清空

并不是真正意义上的清空，缓冲区的数据依然保留着，只是 **重置指针** $\text{position}=0$ 、 $\text{limit}=\text{capacity}$ 、 $\text{mark}=-1$

2. **Buffer flip()**: 反转

将缓冲区的工作状态 "**从写变为读**": 读取刚刚写入的数据、或者重新读取

重置指针，调整边界: $\text{position}=0$ 从初始位置开始读取、 $\text{limit}=\text{position}$ 要读取的有效数据量、 $\text{mark}=-1$

通常在我们从通道 (Channel) 读取数据到缓冲区后，或者在填充完缓冲区准备将其内容写入通道时，都会调用 `flip()` 方法来切换读写模式并调整相应的边界

3. **Buffer rewind()**: 倒退到原点

从头读到尾 $\text{position}=0$ 、 $\text{mark}=-1$

4. **Buffer reset()**: 倒退到标记处 `mark`

将当前位置 `position` 转到以前设置的 `mark` 所在的位置，**从标记处开始读** $\text{position}=\text{mark}$

5. **boolean hasRemaining()**: 判断缓冲区中是否还有元素
(有效元素)

int remaining(): 还剩下多少个元素, $\text{remaining} = \text{limit} - \text{position}$

6. **int capacity()**: 容量

7. **int limit()**: 界限 **Buffer limit(int n)**

8. **int position()**: 当前位置 **Buffer position(int n)**

9. **Buffer mark()**: 标记当前位置

10. **boolean isDirect()**: 是否是直接内存

通常来说: $0 \leq \text{mark} \leq \text{position} \leq \text{limit} \leq \text{capacity}$

```
public abstract class Buffer {
```

```
    public Buffer clear() {  
        position = 0;  
        limit = capacity;  
        mark = -1;  
        return this;  
    }
```

```
    public Buffer flip() {  
        limit = position;  
        position = 0;
```

```
mark = -1;  
return this;  
}
```

```
public Buffer rewind() {  
    position = 0;  
    mark = -1;  
    return this;  
}
```

```
public Buffer reset() {  
    int m = mark;  
    if (m < 0)  
        throw new InvalidMarkException();  
    position = m;  
    return this;  
}
```

```
public final int remaining() {  
    return limit - position;  
}
```

```
public final boolean hasRemaining() {  
    return position < limit;  
}
```

```
public Buffer mark() {  
    mark = position;  
    return this;  
}
```

```
}
```


flip() 用法

```
Buffer buff = new Buffer();  
Channel in = new Channel();  
Channel out = new Channel();  
// 写入数据  
buff.put("header");  
in.read(buff);  
// 反转  
buff.flip();  
// 读取数据  
out.write(buff);
```

ByteBuffer/DirectByteBuffer

Buffer 有很多子类，根据 存储数据类型 的不同，通常有以下几种

- ByteBuffer: I/O 操作、文件读写、网络操作、OpenGL 使用它来传递顶点数据或纹理数据
- CharBuffer
- ShortBuffer
- IntBuffer
- LongBuffer
- FloatBuffer
- DoubleBuffer

这里以 *ByteBuffer* 为例

操作数据相关方法

1. `get()` : 读取单个字节
2. `get(byte[] dst)`: 批量读取多个字节到 `dst` 中
3. `get(int index)`: 读取指定索引位置的字节 (不会移动 position) 放入数据到 Buffer 中
4. `put(byte b)`: 将给定单个字节写入缓冲区的当前位置
5. `put(byte[] src)`: 将 `src` 中的字节写入缓冲区的当前位置
6. `put(int index, byte b)`: 将指定字节写入缓冲区的索引位置 (不会移动 position)

```
public abstract class ByteBuffer
    extends Buffer
    implements Comparable<ByteBuffer>
{

    final byte[] hb;                // 字节缓冲区 Heap Byte
    final int offset;               // 当前偏移位置

    public static ByteBuffer allocateDirect(int capacity) {
        DirectByteBuffer.MemoryRef memoryRef = new
        DirectByteBuffer.MemoryRef(capacity);        // 申请 native 端的内存
        return new DirectByteBuffer(capacity, memoryRef);
    }
    // DirectByteBuffer
}
```

```
}
```

创建

ByteBuffer 有 2 种创建方式：

1. `allocate()`：创建一个 "堆内存"
2. `allocateDirect()`：创建一个 "直接内存/Native内存"
`DirectByteBuffer`

它是通过 `DirectByteBuffer.MemoryRef` 来申请 native 端的内存

直接内存在执行 I/O 操作的时候，效率更高，但是创建的代价也更高

```

public class DirectByteBuffer extends MappedByteBuffer
implements DirectBuffer {

    int address;                // 缓冲区的内存地址

    DirectByteBuffer(int capacity, MemoryRef memoryRef) {
        super(-1, 0, capacity, capacity, memoryRef.buffer,
memoryRef.offset);
        this.memoryRef = memoryRef;
        this.address = memoryRef.allocatedAddress +
memoryRef.offset;            // 缓存数组的内存地址
        cleaner = null;
        this.isReadOnly = false;
    }
}

```

直接内存 (Direct Memory) 是指在 Java 堆外的、直接向系统申请的内存区间。

1. 直接内存不受 Java 虚拟机管理:

它不是虚拟机运行时数据区的一部分，也不是 Java 虚拟机规范中定义的内存区域。

直接内存的分配不会受到 java 堆大小的限制，但是既然是内存，则肯定还是会受到本机总内存的大小及处理器寻址空间的限制。

2. 性能更高:

直接内存/字节缓冲区 "可以避免直接内存到堆内存的数据拷贝过程"，从而可以实现零拷贝，效率更高；

但是另一方面，直接内存通常比堆内存的缓冲区 有更高的分配和释放成本。

标准读写操作

```
public void test2(){
    String str = "itheima";

    // 创建
    ByteBuffer buf = ByteBuffer.allocate(1024);

    // 1. 写
    buf.put(str.getBytes());

    // 转换为读模式
    buf.flip();

    // 2. 读
    byte[] dst = new byte[buf.limit()];
    buf.get(dst, 0, 2);
    System.out.println(new String(dst, 0, 2));           // it
    System.out.println(buf.position());                  // 当前下一个
    读取的位置为2

    // 3. 标记
    buf.mark();

    buf.get(dst, 2, 2);
    System.out.println(new String(dst, 2, 2));           // he
```

```
System.out.println(buf.position());           // 当前下一个
读取的位置为4

// 恢复到 mark 的位置
buf.reset();
System.out.println(buf.position());           // 2

// 4. 判断缓冲区中是否还有剩余数据
if(buf.hasRemaining()){
    System.out.println(buf.remaining());       // 5: 返回
position 和 limit 之间的元素个数
}
}
```

MappedByteBuffer

MappedByteBuffer **映射内存**：用来实现 **内存映射文件** (Memory Mapped File) 的概念

- 内存映射文件：将文件的一部分/全部映射到内存中，使得应用程序通过可以 "**直接操作内存来访问文件**"
- 内存映射文件，适合 "**大文件**" 的操作，对于 "小文件"，映射开销可能大于直接读取造成的开销

我们可以通过 **FileChannel.map(MapMode, int start, int length)** 来创建一块映射内存

- mapMode: 映射方式 READ_ONLY、READ_WRITE、PRIVATE
- start: 映射的起始偏移
- length: 映射的长度

当文件发生修改后，应及时调用 `MappedByteBuffer.force()` 方法，将内存中的数据同步到磁盘，否则可能会由于系统崩溃而丢失数据

```
FileChannel channel = ...;  
MappedByteBuffer mappedByteBuffer =  
channel.map(FileChannel.MapMode.READ_WRITE, 0,  
channel.size());
```

Channel 通道

Channel **数据读写通道**: 表示一个进行数据读写的通道，它提供了一种方式来连接到各种类型的 I/O 目标源（例如 文件 File、套接字 Socket、管道 Pipeline 等等）

Channel 对应传统 I/O 模型中的 "**流 Stream**"，相比 Stream 它有以下特性

1. 双向通讯

流 Stream 是单向通讯，分为 InputStream/OutputStream，只能读/或者写

通道 Channel 是双向通讯，能同时进行读写操作
read()/write()

2. 同步非阻塞

流 Stream 是同步阻塞的，读写会阻碍主线程

InputStream.read() 如果没有数据可读，会一直阻塞等待，直到有数据

OutputStream.write() 如果缓冲区满了，会一直阻塞等待，直到有空余位置

通道 Channel 也是同步的，但是它的读写操作不会阻碍主线程：

Channel.read() 如果没有数据可读，它不会一直等待，而是直接返回一个结果状态（如 -1 表示 EOF，0 表示目前无数据可读）；

Channel.write() 如果缓冲区满了，它也不会阻塞，而是直接返回一个结果状态表示操作未能完成

3. 多路复用

结合 Selector 一起使用，可以实现多路复用：一个线程同时处理多个 Channel

4. 缓冲区

通道 Channel 本身不能直接访问数据，所有数据必须通过 "缓冲区 Buffer" 完成

read(Buffer) 将数据读取到 Buffer 中，
write(Buffer) 写入 Buffer 中的数据

Channel

Channel 就是一个很简单的接口，但是它有很多种类

- WritableByteChannel
- ReadableByteChannel
- GatheringByteChannel 聚集写
- ScatteringByteChannel 分散读
- SelectableChannel 可以多路复用的

```
public interface Channel extends Closeable {  
    public boolean isOpen();  
    public void close() throws IOException;  
}
```

FileChannel

常用的 Channel 实现类

- **FileChannel**：用于读取、写入、映射和操作文件的通道。
- **DatagramChannel**：通过 UDP 读写网络中的数据通道。

- **SocketChannel**：通过 TCP 读写网络中的数据。
- **ServerSocketChannel**：可以监听新进来的 TCP 连接，对每一个新进来的连接都会创建一个 SocketChannel
ServerSocketChannel 类似 ServerSocket，SocketChannel 类似 Socket

这里以 *FileChannel* 为例

创建 Channel：调用通道的 **FileChannel.open()** 方法即可获取，或者调用对应的 **FileInputStream.getChannel()** 获取

- FileInputStream
- FileOutputStream
- RandomAccessFile
- DatagramSocket
- Socket
- ServerSocket
- Files.newByteChannel(). 获取字节通道
- FileChannel.open(). 获取指定通道

常用方法

- **int read(ByteBuffer dst)**：从 Channel 中读取数据到

ByteBuffer (对于 ByteBuffer 来说是 write)

- `long read(ByteBuffer[] dsts)`: 将 Channel 中的数据 “分散 Scatter” 到 ByteBuffer[] 数组中
- `int write(ByteBuffer src)`: 将 ByteBuffer 中的数据写入到 Channel (对于 ByteBuffer 来说是 read)
- `long write(ByteBuffer[] srcs)`: 将 ByteBuffer[] 中的数据 “聚集 Gather” 到 Channel 中
- `void close()`: 关闭
- `long transferFrom(ReadableByteChannel src, long position, long count)`: 复制通道数据
- `long transferTo(long position, long count, WritableByteChannel target)`: 复制通道数据到

- `long size()`: 返回此通道的文件的当前大小

- `long position()`: 返回此通道的文件位置

`FileChannel position(long p)`: 设置此通道的文件位置

- `FileChannel truncate(long s)`: 将此通道的文件截取为给定大小

- `void force(boolean metaData)`: 强制将所有对此通道的文件更新写入到 “底层存储设备” 中

标准的 Channel-Buffer 读写文件

写文件

```
// 1、File
FileOutputStream fos = new
FileOutputStream("E:\\test\\data01.txt");

// 2. Channel
FileChannel channel = fos.getChannel();

// 3. Buffer
ByteBuffer buffer = ByteBuffer.allocate(1024);

// 4. 写
for (int i = 0; i < 10; i++) {
    buffer.clear();                // 清空缓冲区
    buffer.put("hello".getBytes());
    buffer.flip();                 // 切换读模式
    channel.write(buffer);         // 将缓冲区的数据写入到文件通道
}

// 5. 关闭
channel.close();
```

读文件

Charset.decode(ByteBuffer) 使用指定编码读取 Buffer 中的数据

```
// 1、File
FileInputStream is = new
FileInputStream("E:\\test\\data01.txt");
```

// 2、Channel

```
FileChannel channel = is.getChannel();
```

// 3、Buffer

```
int bufferSize = 1024 * 1024;
```

```
ByteBuffer buffer = ByteBuffer.allocate(bufferSize);
```

```
ByteBuffer bb = ByteBuffer.allocate(1024);           // 每一行  
的 Buffer
```

// 4、读

```
int bytesRead = channel.read(buffer);                // 对
```

Buffer 来说是 write

```
while (bytesRead != -1) {
```

```
    buffer.flip();                                     // 切换模
```

式, 写->读

// 读取一行

```
while (buffer.hasRemaining()) {
```

// 先判断

```
    byte b = buffer.get();
```

```
    if (b == 10 || b == 13) {
```

// 换行或回车

```
        bb.flip();
```

// 切换读模式

```
        final String line = Charset.forName("utf-
```

```
8").decode(bb).toString();                // 读一行
```

```
        System.out.println(line);
```

```
        bb.clear();
```

// clear

```
    } else {
```

```
        if (bb.hasRemaining())                // 先判断
```

```
            bb.put(b);                        // 写入行
```

```
        else {
```

```

        bb = reAllocate(bb);           // 空间不够扩容
        bb.put(b);
    }
}

buffer.clear();                       // clear
bytesRead = channel.read(buffer);     // 继续
读: 往 buffer 中写
}

// 5. 关闭
channel.close();

```

文件复制

我们也可以直接使用 `transferTo()` / `transferFrom()` 操作

```

// 1. File
File srcFile = new File("E:\\test\\Aurora-4k.jpg");
File destFile = new File("E:\\test\\Aurora-4k-new.jpg");
FileInputStream fis = new FileInputStream(srcFile);
FileOutputStream fos = new FileOutputStream(destFile);

// 2. Channel
FileChannel isChannel = fis.getChannel();
FileChannel osChannel = fos.getChannel();

// 3. Buffer
ByteBuffer buffer = ByteBuffer.allocate(1024);

// 4. 复制
while(isChannel.read(buffer)>0){
    buffer.flip();
}

```

```
osChannel.write(buffer);  
buffer.clear();  
}
```

// 5. 关闭

```
isChannel.close();  
osChannel.close();
```

聚集/扩散操作

// 1. File

```
RandomAccessFile raf1 = new RandomAccessFile("1.txt", "rw");
```

// 2. Channel

```
FileChannel channel1 = raf1.getChannel();
```

// 3. Buffer

```
ByteBuffer buf1 = ByteBuffer.allocate(100);  
ByteBuffer buf2 = ByteBuffer.allocate(1024);  
ByteBuffer[] bufs = {buf1, buf2};
```

// 4. 分散读取

```
channel1.read(bufs);
```

```
for (ByteBuffer byteBuffer : bufs) {  
    byteBuffer.flip();  
}
```

```
System.out.println(new String(bufs[0].array(), 0,  
bufs[0].limit()));          // 0~limit
```

```
System.out.println("-----");
```

```
System.out.println(new String(bufs[1].array(), 0,  
bufs[1].limit()));
```

```
// 5. 聚集写入
```

```
RandomAccessFile raf2 = new RandomAccessFile("2.txt", "rw");  
FileChannel channel2 = raf2.getChannel();  
  
channel2.write(bufs);
```

Selector 选择器/多路复用器

Selector 选择器/多路复用器： `SelectableChannel` 对象的多路复用器，使用一个 **单独的线程** 管理多个 Channel，是 **非阻塞 IO** 的核心

Selector 工作原理：

1. **注册**：应用程序将 `SelectableChannel` 注册到 Selector 上，并指定它感兴趣的事件类型
2. **阻塞**：Selector 调用 `select()` 方法，阻塞等待事件发生
3. **准备队列**：当事件发生时，Selector 会将其添加到一个准备队列中 `SelectionKey`
4. 应用程序可以遍历准备队列，并处理每个事件

Selector 可以监控的 **事件类型**：`Channel.register(Selector, int )`

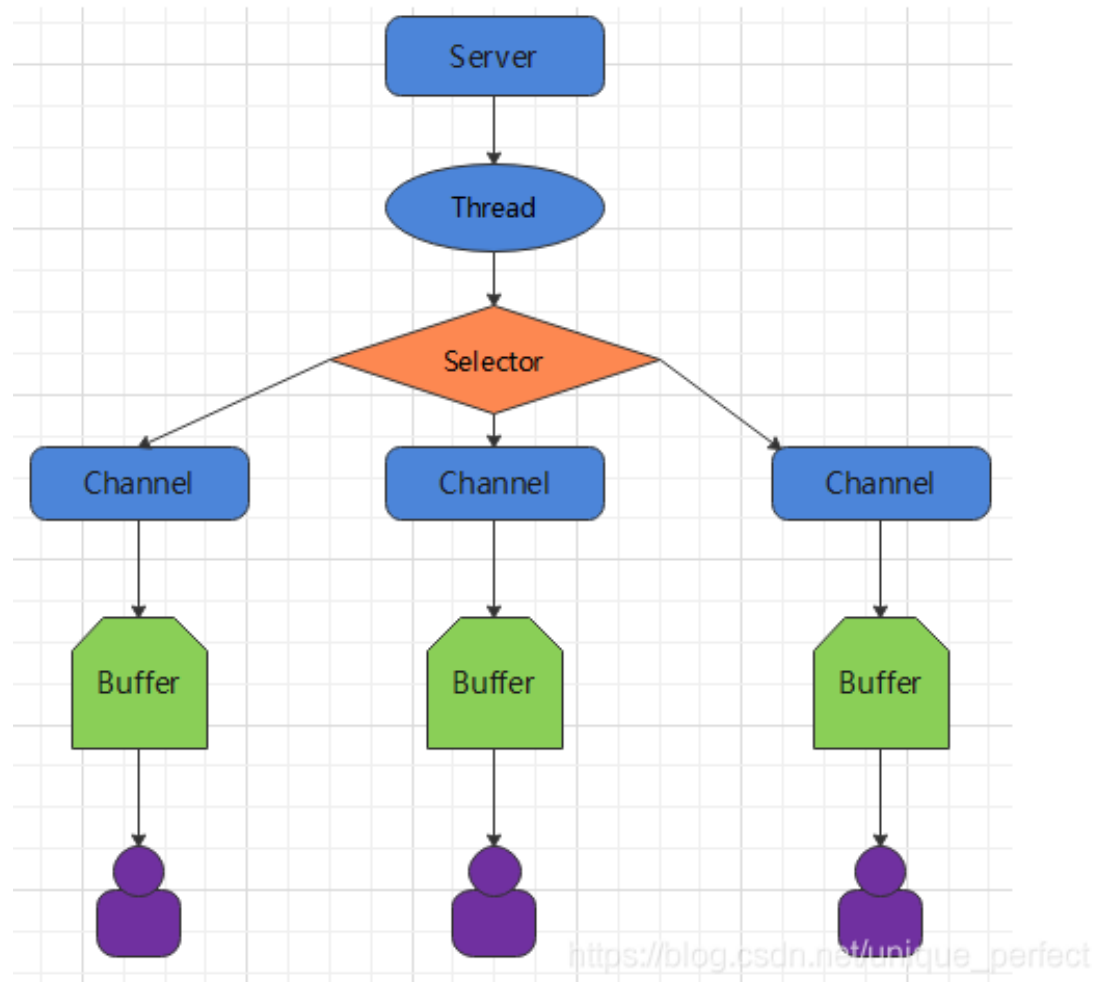
- **OP_ACCEPT**: 连接请求到达
- **OP_READ**: 数据可读
- **OP_WRITE**: 数据可写
- **OP_CONNECT**: 连接完成

Selector 可以同时监控多个 SelectableChannel (注册的通道) 的 IO 状况

它能够 "监听" 所有 "注册的通道上" 是否有事件发生, 如果有事件 (读写) 发生, 便获取事件然后派发给各个注册的 Channel

Selector 优点

1. **并发处理**: 可以同时监控多个 Channel, 而是为每个连接都创建一个线程, 不用去维护多个线程
2. **节约资源**: 只处理就绪的 Channel, 避免了资源浪费



// 1. Channel

```
ServerSocketChannel ssChannel = ServerSocketChannel.open();  
ssChannel.configureBlocking(false); // 切换  
非阻塞模式  
ssChannel.bind(new InetSocketAddress(9898)); // 绑定  
端口
```

// 2. Selector

```
Selector selector = Selector.open();
```

// 3. 注册

```
ssChannel.register(selector, SelectionKey.OP_ACCEPT); // 将通  
道注册到选择器上, 并且指定 “监听接收事件”
```

Selector

创建 `open()`

准备队列 `SelectionKeys`

- `keys()`：所有 key 值，不能更改，否则会抛出错误
- `selectedKeys()`：已选择的 key 值，只能 remove 不能 add，否则会抛出错误

阻塞等待事件发生 `select()`

- 返回值：可用的 Channel 数量
- 这是一个 **阻塞方法**：它会一直等待，直到有可用的 Channel 为止

如果 当前线程中断、或者 调用了 `wakeup()` 方法，也会立即返回，因此它的返回值可能是 0

非阻塞方法 `selectNow()`：立即返回当前可用的通道 Channel 数量，非阻塞方法

`Selector.select()` 是一个阻塞方法，但是为什么任然叫 NIO 为非阻塞 I/O 模型？

`Selector.select()` 虽然是一个阻塞方法，但这并不意味着 NIO 是一个阻塞式 I/O

因为 `Selector.select()` 它是阻塞等待事件发生，而不是阻塞等待 I/O 操作完成

- 如果没有任何事件发生，那么 `Selector.select()` 会阻塞线程，直到有事件到来
- 但是如果一旦有事件发生，那么 `Selector.select()` 会立刻返回

```
public abstract class Selector implements Closeable {

    public static Selector open() throws IOException {
        return SelectorProvider.provider().openSelector();
    }

    // keys()
    public abstract Set<SelectionKey> keys();
    // 所有 key 值，这个返回的 Set 集合是不能更改的，否则会抛出错误
    public abstract Set<SelectionKey> selectedKeys();
    // 已选择的 key 值，只能 remove 不能 add

    // select()
    public abstract int selectNow() throws IOException;
    // 非阻塞方法
    public abstract int select() throws IOException;
    // 阻塞方法
    public abstract int select(long timeout) throws
    IOException;           // 最多阻塞多少时间，0 是无限阻塞

    public abstract Selector wakeup();

    public abstract void close() throws IOException;
```

```
}
```

事件 SelectionKey

Selector 可以监听的 **事件类型** **SelectionKey**

- 读 : `SelectionKey.OP_READ`
- 写 : `SelectionKey.OP_WRITE`
- 连接 : `SelectionKey.OP_CONNECT`
- 接收 : `SelectionKey.OP_ACCEPT`

另外 `SelectionKey` 还可以代表我们注册的 **通道 key 值**

- **`selector()`** : 对应的选择器
 - **`channel()`** : 对应的通道
 - **`readyOps()`** : 对应的事件类型 (注册时候)
 - **`isAcceptable()`** : 是否可以接受 `accept()` 事件
- `isConnectable()`**
- `isWritable()`**
- `isReadable()`**

```
public abstract class SelectionKey {
```

```
public abstract Selector selector();
public abstract SelectableChannel channel();

public abstract int readyOps();

public final boolean isAcceptable() {
    return (readyOps() & OP_ACCEPT) != 0;
}

public final boolean isConnectable() {
    return (readyOps() & OP_CONNECT) != 0;
}

}
```

通道 SelectableChannel

Selector 要求注册的通道是 **SelectableChannel** 类型，**ServerSocketChannel** 就是一个典型的 SelectableChannel

- **configureBlocking()**: 是否为阻塞模式，一般在多路复用模式下为 false
- **bind(InetSocketAddress)**: 绑定端口号
- **register(Selector sel, int ops)**: 注册到 Selector 上

标准的选择器

服务器端流程

如果使用 BIO 那么就是来一个 Socket 开一个线程

而这里的 NIO 只开了一个线程，并且只有当需要读写的时候，才去操作对应的 SocketChannel

```
public class Server {  
  
    public static void main(String[] args) throws Exception {  
        // 创建一个ServerSocketChannel, 并绑定到端口 8080  
        ServerSocketChannel serverSocketChannel =  
ServerSocketChannel.open();  
        serverSocketChannel.configureBlocking(false);  
        // 切换为非阻塞模式  
        serverSocketChannel.bind(new  
InetSocketAddress(8080));           // 绑定到端口  
  
        // 创建一个Selector  
        Selector selector = Selector.open();  
  
        // 将ServerSocketChannel注册到Selector上, 并监听OP_ACCEPT事件  
        serverSocketChannel.register(selector,  
SelectionKey.OP_ACCEPT);  
  
        while (true) {  
            // 阻塞等待事件发生  
            // 😊 Selector.select() 虽然是一个阻塞方法, 但这并不意味着 NIO  
            是一个阻塞式 I/O
```

```
// 因为 Selector.select() 它是阻塞等待事件发生，而不是阻塞等待
I/O 操作完成

// 如果没有任何事件发生，那么 Selector.select() 会阻塞线程，直
到有事件到来

// 但是如果一旦有事件发生，那么 Selector.select() 会立刻返回
selector.select();

// 遍历所有已就绪的SelectionKey
Set<SelectionKey> selectedKeys =
selector.selectedKeys();
for (SelectionKey selectedKey : selectedKeys) {
    // 判断事件类型
    if (selectedKey.isAcceptable()) {
// 客户端连接

        SocketChannel socketChannel =
serverSocketChannel.accept();
        socketChannel.configureBlocking(false);
        // 将SocketChannel注册到Selector上，开始监听

OP_READ事件

        socketChannel.register(selector,
SelectionKey.OP_READ);

    } else if (selectedKey.isReadable()) {
// 客户端发送数据

        SocketChannel socketChannel =
(SocketChannel) selectedKey.channel();
        // 读取数据
        ByteBuffer buffer =
ByteBuffer.allocate(1024);
        int len = socketChannel.read(buffer);
```



```

        System.out.println(new
String(buffer.array(), 0, len));

        // 将SocketChannel重新注册到Selector上, 继续监听
OP_READ事件

        socketChannel.register(selector,
SelectionKey.OP_READ);
    }
}

// 清除已处理的SelectionKey
selectedKeys.clear();
}
}
}
}

```

客户端流程

传输的数据用 ByteBuffer 表示

```

// 1. SocketChannel
SocketChannel socketChannel = SocketChannel.open(new
InetSocketAddress("127.0.0.1", 8888));
socketChannel.configureBlocking(false);

// 2. Buffer
ByteBuffer buffer = ByteBuffer.allocate(1024);

// 3. 写
Scanner scanner = new Scanner(System.in);
while (true){
    System.out.print("请输入:");

```

```
String msg = scanner.nextLine();  
buffer.put(msg.getBytes());  
buffer.flip(); // 类似 flush()  
socketChannel.write(buffer);  
buffer.clear();  
}
```